

# Books & Records Provider Integration

## Architecture Decision Document

Revision 2.0 · Status: Draft for review  
Prepared for: Chris Hahn · Date: 27 July 2026  
*Supersedes Revision 1.0 (27 July 2026)*

**Reading this document.** Every quantitative value that has not yet been confirmed with the provider or with capacity planning is rendered in **bold orange** and carries a **PH-nn** identifier. These are proposed defaults chosen to make the design reviewable — they are not measured or agreed figures. Section 14 lists every one of them in a single register. No placeholder may survive into an approved revision.

# 1. Purpose and Revision Note

This document defines the target architecture for integrating enterprise domain APIs with the third-party books and records provider. It captures the design decisions reached in review: a strictly generic, shared integration layer; a clear ownership boundary between that layer and each domain team’s data knowledge; and the inbound and outbound data-exchange mechanics needed to support five distinct provider channels reliably in both directions.

## 1.1 What changed in Revision 2.0

Revision 1.0 described the design but did not state what it was designed to achieve, and three mechanisms in it did not hold under failure. This revision addresses both. The material changes are:

| Change                                                                                                            | Reason                                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Added a non-functional requirements and SLO section (§3).                                                         | Revision 1.0 contained no availability target, no throughput figure, and no recovery objective. Every resilience parameter in the design is unresolvable without them.                                                   |
| Added an explicit ordering and version model (ADR-003).                                                           | Full-file snapshots and CDC events had no common ordering rule, so a snapshot could silently overwrite newer change data. Revision 1.0 deferred this as an open question; it is foundational.                            |
| Corrected outbound dispatch with a sticky-pending rule (ADR-005).                                                 | Partitioning the pending queue by entity key preserved order only within the queue. A half-open breaker allowed a newer write to overtake a still-draining older one for the same entity.                                |
| Replaced naive dead-lettering with ordered-stream quarantine (ADR-007).                                           | Skipping a poisoned message on an ordered stream applies every later event to a wrong base state, and a dead-lettered outbound write left the domain serving an unbooked value as clean.                                 |
| Added degraded-mode data classification (ADR-004).                                                                | The requirement to remain available irrespective of provider availability, applied uniformly, mandates serving unmarked stale balances. For some data subjects that is a misrepresentation rather than a resiliency win. |
| Gateway topology decided explicitly and deployed as per-LOB cells (ADR-001).                                      | Revision 1.0 asserted a shared runtime without considering the alternative, and without analysing the correlated failure domain it creates across every line of business.                                                |
| Rebuilt the security section around entitlements, information barriers, retention and credential lifecycle (§11). | Revision 1.0 covered transport security only. A single component holding every LOB’s credentials and data raises segregation and records-retention questions that were not addressed.                                    |
| Reclassified two “open questions” as blocking discovery items (§13).                                              | Provider idempotency semantics and per-instance SLAs are not open questions — the design branches on the first and every threshold in it depends on the second.                                                          |

## 2. Problem Statement

- Multiple enterprise APIs, in distinct data domains, need to publish and consume data from a third-party books and records provider.
- The provider runs a separate server instance per line of business (LOB).
- Enterprise APIs are heavily dependent on the provider but must remain able to serve enterprise experiences (UI, mobile) when the provider is unavailable, **subject to the per-data-subject degraded-mode policy in ADR-004**. Availability is not unconditionally the correct behaviour for every data subject.
- All domain APIs are *believed* to share the same integration needs — data exchange, scalability, resiliency and security. This is the sole justification for a shared layer and is treated in this revision as an assumption to be validated per domain (DI-06), not as a given.

### 2.1 Scope

In scope: the integration layer between enterprise domain APIs and the provider, the inbound and outbound data paths across all five provider channels, and the cross-cutting resiliency, security and observability concerns of that layer.

Out of scope: the internal design of any individual domain API beyond the obligations this document places on it; the enterprise experience tier; and migration of existing point-to-point integrations, which is tracked separately as FU-09.

### 3. Non-Functional Requirements and Service Level Objectives

Revision 1.0 stated a target of 99.9% availability and “high throughput”. Neither is testable as written. A single composite availability number is also the wrong shape for this system: the read path and the write path are served by different mechanisms, fail independently, and can sustain different targets. This section replaces both with measurable objectives.

#### 3.1 Volumetrics

All figures below are placeholders pending capacity planning and provider confirmation. They are stated so that queue sizing, partition counts, pool sizes and breaker thresholds have something concrete to be derived from, and so that a load test has a target.

| Dimension                      | Value                                               | Notes                                                                                                          |
|--------------------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Inbound CDC — sustained        | <b>1,200 events/sec</b> (PH-01)                     | Aggregate across all LOB instances.                                                                            |
| Inbound CDC — peak             | <b>6,000 events/sec</b> (PH-02)                     | Peak-to-average ratio of 5× at market open and close. This ratio, not the average, sizes consumer parallelism. |
| Outbound writes — sustained    | <b>150 intents/sec</b> (PH-03)                      | Aggregate across domains.                                                                                      |
| Outbound writes — peak         | <b>900 intents/sec</b> (PH-04)                      | Sizes the pending queue write path and the drain rate limiter.                                                 |
| EOD full file                  | <b>8M rows / ~4 GB per LOB</b> (PH-05)              | Drives whether the domain-side diff can be in-memory or must be streaming. See §9.4.                           |
| EOD delta file                 | <b>200k rows per LOB</b> (PH-06)                    |                                                                                                                |
| Batch window                   | <b>22:00–04:00 ET, complete by 05:30 ET</b> (PH-07) | Hard deadline. Overrun is an incident, not a delay.                                                            |
| Payload size                   | <b>p50 2 KB, p99 40 KB</b> (PH-08)                  |                                                                                                                |
| Domain API read latency budget | <b>p99 ≤ 200 ms</b> (PH-09)                         | Served entirely from the domain-local store; no synchronous provider call is on this path.                     |
| Growth allowance               | <b>3× over 3 years</b> (PH-10)                      | Design must absorb this without re-architecture.                                                               |

#### 3.2 Availability objectives

Stated per path, because they are delivered by different mechanisms. Provider availability is explicitly not inherited: these targets apply to the enterprise surface regardless of the provider’s own uptime.

| Objective                                         | Target                | Mechanism                                                                                  |
|---------------------------------------------------|-----------------------|--------------------------------------------------------------------------------------------|
| Read availability — market hours (06:00–20:00 ET) | <b>99.95%</b> (PH-11) | Domain-local store; the read path does not traverse the gateway or the provider (ADR-002). |

| Objective                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Target                                                     | Mechanism                                                                                                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Read availability — outside market hours                                                                                                                                                                                                                                                                                                                                                                                                                                                | <b>99.9%</b> (PH-12)                                       | As above.                                                                                                                      |
| Write acceptance availability                                                                                                                                                                                                                                                                                                                                                                                                                                                           | <b>99.9%</b> (PH-13)                                       | Transactional outbox: acceptance depends only on the domain's own datastore, not on the gateway or the provider (ADR-005).     |
| Write confirmation — provider available                                                                                                                                                                                                                                                                                                                                                                                                                                                 | <b>p99 ≤ 5 s to ACK</b> (PH-14)                            | Direct send.                                                                                                                   |
| Freshness — CDC path                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>p99 ≤ 5 s provider-commit to domain-visible</b> (PH-15) | This is the SLI that actually matters to a user looking at a screen. Availability without freshness is not a useful guarantee. |
| Freshness — file path                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | <b>within batch window</b> (PH-07)                         |                                                                                                                                |
| <b>Note on the original 99.9% target.</b> 99.9% is approximately 44 minutes of downtime per month. For customer-facing financial data during market hours this is low relative to industry practice, and it is achievable here because the read path does not depend on the provider. The higher market-hours target in PH-11 reflects what the architecture can actually deliver rather than what was originally asked for; it should be confirmed against business appetite and cost. |                                                            |                                                                                                                                |

### 3.3 Definition of an outage

An availability target that is not accompanied by a definition of failure cannot be measured, and cannot be defended in an incident review. For the purposes of §3.2:

- **Read outage:** the domain API returns 5xx, times out beyond the latency budget (PH-09), or fails closed under ADR-004, for more than **1%** of requests to a given data subject over a **5-minute** window (PH-16).
- **Write outage:** the domain API cannot durably accept and commit an outbound intent. Note that a provider outage is *not* a write outage — intents are still accepted — but it does start the confirmation clock in §3.5.
- **Freshness breach:** store age exceeds the data subject's class threshold (ADR-004). Tracked against its own error budget, separately from availability, because a fresh-but-down system and a up-but-stale system fail customers in different ways.

Planned provider maintenance windows are excluded from the freshness budget provided they are notified in advance and registered in the gateway's maintenance calendar (§10.4). They are not excluded from read availability, which must survive them by design.

### 3.4 Recovery objectives

| Component                     | RTO                   | RPO                | Notes                                                                                                                                                                                        |
|-------------------------------|-----------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Domain-local projection store | <b>30 min</b> (PH-17) | <b>rebuildable</b> | Derived state. Recovered by replay/backfill (§9.6) rather than by backup restore, but the replay must be fast enough to meet the RTO — this is what makes backfill a first-class requirement |

| Component                      | RTO                   | RPO      | Notes                                                                                                                                      |
|--------------------------------|-----------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------|
|                                |                       |          | rather than an afterthought.                                                                                                               |
| Outbox (in domain datastore)   | <b>15 min</b> (PH-18) | <b>0</b> | An accepted write must never be lost. Synchronous replication required.                                                                    |
| Pending queue and send-log     | <b>15 min</b> (PH-19) | <b>0</b> | Same reasoning. These are the stateful components and they are the hard part of DR; treating them as ordinary services would be a mistake. |
| Gateway (stateless components) | <b>5 min</b> (PH-20)  |          | Multi-AZ, active-active per cell.                                                                                                          |

### 3.5 Write confirmation obligation

Durability is not the same as completion. When the provider is unavailable, the domain has accepted an intent that the system of record has not yet booked. That gap has to have a stated limit, because a customer who believes a transaction is booked when it is not is exposed, and so is the firm.

| Unconfirmed age           | Required action                                                                                                                                         |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>15 minutes</b> (PH-21) | Record is marked pending-confirmation in the domain store; enterprise experiences must surface this state rather than presenting the write as complete. |
| <b>4 hours</b> (PH-22)    | Operational escalation; business owner notified; customer communication assessed.                                                                       |
| <b>24 hours</b> (PH-23)   | Automatic escalation to the break-management process (§12) and treatment as a confirmed break rather than a delay.                                      |

## 4. Guiding Principle: Generic Gateway, Domain-Owned Data

The integration layer between domain APIs and the provider is built and owned as a shared, generic capability. It handles connectivity, delivery guarantees, security and routing — the concerns that are identical across every domain. It deliberately does not hold data shape, business meaning, parsing logic or validation rules for any domain.

This boundary is intentional. An integration layer that accumulates domain data knowledge over time is how enterprise service buses become the anti-pattern they are known for: every domain change requires touching shared code, a team's data expertise ends up living in a component it does not own, and the "shared" layer becomes the bottleneck every team routes through. Keeping the gateway a dumb pipe with smart delivery guarantees, and treating each domain team as the smart, authoritative endpoint for its own data, avoids that outcome and lets domains evolve their schemas and business rules independently.

### 4.1 The practical split

| Concern                        | Gateway (generic)                                                               | Domain API (data-aware)                                   |
|--------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------|
| Protocol / transport           | File transfer, Kafka consumption, REST/TCP clients, Data Exchange client        | None                                                      |
| Message framing & integrity    | Checksums, byte/row counts against the manifest, well-formedness, TCP framing   | Supplies the manifest convention as configuration (DI-03) |
| Envelope metadata              | Source, LOB instance, entity key, version, asOf, correlation & idempotency keys | Supplies the declarative extraction expressions (§4.2)    |
| Routing / addressing           | Executes the routing rule against extracted metadata                            | Supplies the routing rule (config, not code)              |
| Parsing & schema mapping       | None beyond declarative key/version extraction                                  | Full ownership: field mapping, canonical model            |
| Business / semantic validation | None                                                                            | Full ownership: business rules, state transitions         |
| Ordering & conflict resolution | Carries version and asOf in the envelope; partitions by entity key              | Applies the monotonic version rule (ADR-003)              |
| Outbound payload construction  | None                                                                            | Full ownership: builds provider-shaped payload            |
| Idempotency classification     | Executes the classification (upsert vs. dedup)                                  | Defines the classification per operation                  |
| Delivery guarantees            | Retry, breaker, bulkhead, sticky-pending, drain, idempotent send-log            | None                                                      |
| Security                       | Auth, encryption, credential management per LOB instance                        | Entitlement decisions on its own data                     |
| Reconciliation execution       | Schedules the job, delivers file + event state                                  | Performs the diff, publishes drift result                 |

## 4.2 Where the boundary bends, and why that is stated rather than hidden

Revision 1.0 claimed the gateway performs no parsing at all. That claim does not survive contact with the design. Three of the gateway's own responsibilities require reaching into the payload: row counts and well-formedness checks, extraction of the idempotency and entity keys for inbound CDC, and routing when a channel carries more than one domain's entities.

Pretending otherwise is how the ESB failure mode actually begins — not with a deliberate decision, but with a small piece of format knowledge quietly added to shared code because the document said it was not supposed to be there. The honest resolution is to make the minimum necessary content awareness explicit, declarative, and domain-owned:

- Each domain registers, as configuration, a set of extraction expressions (JSONPath, XPath, or column index depending on channel) that yield exactly three things: entity key, version, and routing key.
- The gateway evaluates those expressions generically. It has no compiled knowledge of any schema, and adding a domain requires no change to gateway code.
- Anything beyond those three fields remains opaque to the gateway and is the domain's responsibility.

This depends on confirming channel-to-domain cardinality with the provider (DI-04). If every channel maps to exactly one domain, simple table-driven routing suffices and the extraction requirement reduces to entity key and version only. If a channel carries multiple domains — common in books and records feeds — then either this mechanism or a fan-out-to-all model is required, and the fan-out model multiplies inbound load by the number of domains. **This must be resolved before build, not during it.**

## 5. Decision Records

Each decision is recorded with the options considered and the consequences accepted, so that a future reader can tell what was decided from what was merely described, and can revisit a decision when its context changes.

### ADR-001 — Gateway topology: shared service, deployed as per-LOB cells

**Status:** Accepted

#### Context

Revision 1.0 asserted a single shared integration service without considering alternatives. That is the most consequential decision in the architecture and it carries a cost the document did not acknowledge: a shared runtime in the path of every domain, in both directions, is a correlated failure domain spanning every line of business. A bad deploy, a poisoned routing config, an expired certificate or a runaway backlog would affect all of them at once — working directly against the availability target the layer exists to support.

#### Options considered

| Option                                               | Assessment                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Shared library or sidecar, deployed into each domain | Delivers the same uniformity of retry, breaker and envelope logic with no shared runtime and no shared fate. Weaknesses: credentials are distributed rather than held centrally, cross-domain rate limiting against a shared LOB instance becomes a coordination problem, and version skew across domains is hard to control. Rejected primarily on credential custody and the inability to enforce a global per-instance rate limit. |
| Single shared service (Revision 1.0 as written)      | Centralises credentials and per-instance rate limiting, which are its genuine advantages. Rejected as specified because it makes every LOB share one blast radius.                                                                                                                                                                                                                                                                    |
| Shared service deployed as per-LOB cells             | Retains centralised credential custody and per-instance rate-limit enforcement while containing failure, deployment and configuration blast radius to a single line of business. Costs more to operate: N deployments, N monitoring surfaces, N on-call surfaces.                                                                                                                                                                     |
| Per-domain bespoke integration                       | Rejected. Duplicates the hardest and most failure-prone logic in the system N times, and every domain gets it subtly wrong in a different way.                                                                                                                                                                                                                                                                                        |

#### Decision

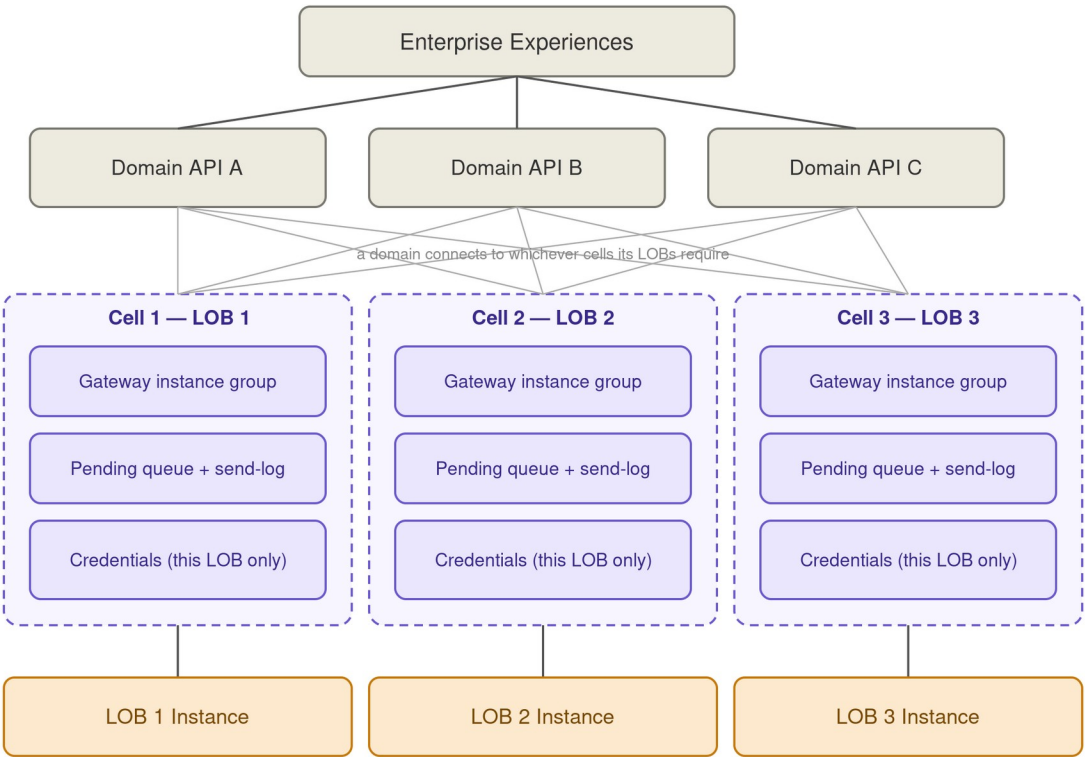
Build the gateway as a shared service, and deploy it as independent cells — one cell per LOB instance. A cell comprises its own gateway instance group, its own pending queue and send-log, and credentials scoped to that LOB alone. No cell shares a runtime, a datastore or a configuration surface with another.

This is also the mechanism by which the shared-layer decision becomes compatible with the availability objectives in §3.2, and it partly addresses the information-barrier concern in §11.2 by removing any single process that holds every LOB's credentials.

#### Consequences

-

- 
- 
- 



Each cell is deployed, configured, rate-limited and rolled back independently. A bad deploy, a poisoned routing config, a credential expiry or a runaway backlog is contained to one line of business rather than to the whole enterprise.

Figure — Cell-based gateway deployment: one cell per LOB instance, no shared runtime across lines of business

Figure 1 — Deployment view: cell-based gateway topology (ADR-001)

## ADR-002 — Domain-local store as the availability mechanism

**Status:** Accepted

### Context

The requirement that enterprise APIs remain available irrespective of provider availability is satisfied by exactly one property of this design: domain APIs serve reads from their own local projection store and never call the provider synchronously on a read path. Revision 1.0 relied on this but never stated it, which meant the load-bearing element of the availability argument was invisible and untested.

### Options considered

| Option                                                   | Assessment                                                                                                                                                                  |
|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synchronous read-through to the provider with a cache    | Rejected. Provider availability becomes enterprise availability at exactly the moment the cache misses, which is the moment it matters.                                     |
| Domain-local projection store, asynchronously maintained | Selected. Decouples read availability from the provider entirely. Introduces staleness, which is a real cost and is managed explicitly by ADR-004 rather than ignored.      |
| Gateway-owned shared cache                               | Rejected. Puts domain data shape into the shared layer, violating §4, and makes the gateway a read-path dependency — the opposite of what the availability target requires. |

### Decision

Each domain owns a local projection of the provider state it needs, maintained asynchronously from the inbound channels. All customer-facing reads are served from it. The gateway is not on the read path at all, so gateway unavailability degrades freshness but not availability.

### Consequences

- 
- 
- 
-

## ADR-003 — Ordering and version model across channels

**Status:** Accepted

### Context

The same entity is updated through more than one channel. A full EOD file is a snapshot as of time T; CDC events with timestamps after T may already have been applied to the domain store. Without a common ordering rule, applying the file-derived events overwrites newer data with older data — and does so silently.

Revision 1.0 recorded this as an open question (“which channel is authoritative when two channels disagree?”). It is not a question that can be left open: reconciliation, which exists to detect drift, would itself become a source of drift.

### Options considered

| Option                                                 | Assessment                                                                                                                                                    |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Channel precedence (e.g. files always win)             | Rejected. Precedence is time-independent, so a stale snapshot still beats a fresh event. It trades a silent race for a systematic error.                      |
| Processing-time last-writer-wins                       | Rejected. Processing time reflects our pipeline, not the provider’s commit order. Retries and backlog drains reorder it freely.                               |
| Monotonic per-entity version sourced from the provider | Selected. Ordering is derived from the system of record, which is the only authority that can settle it, and is stable under replay, retry and backlog drain. |

### Decision

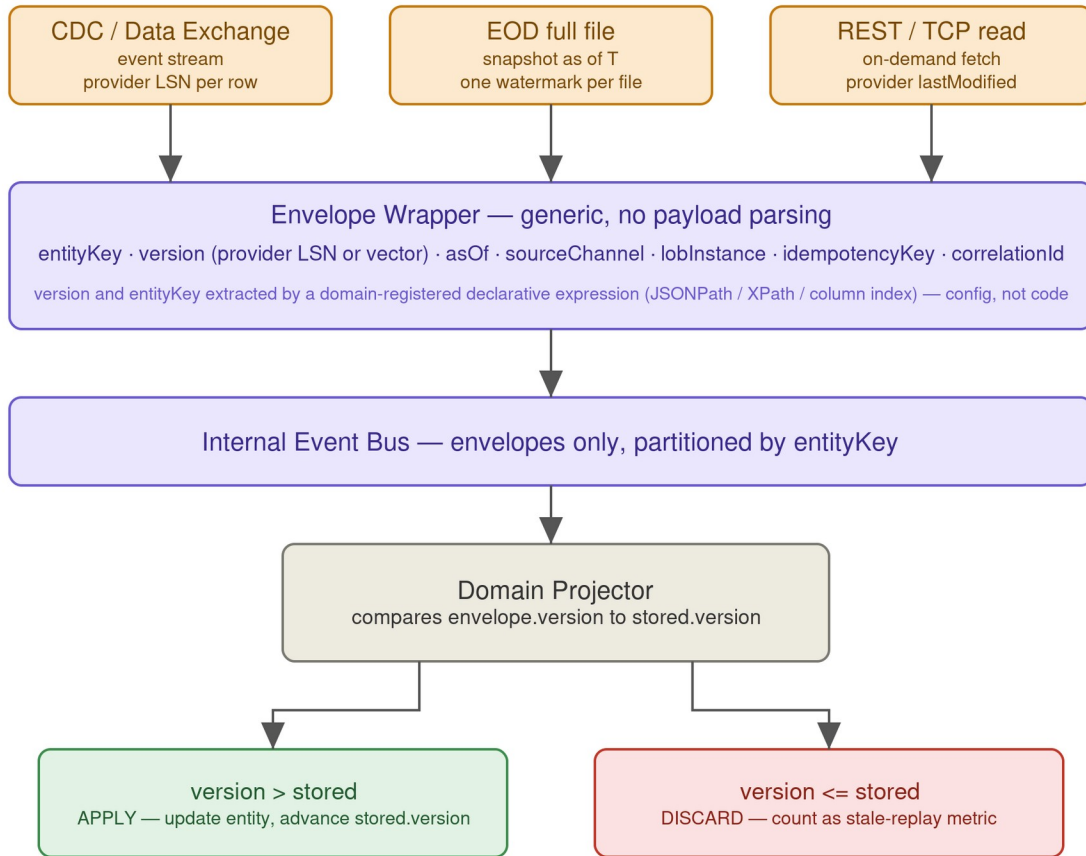
Every envelope carries a monotonic per-entity version taken from the provider — an LSN, sequence number, or provider-side last-modified value. Domain projectors apply an event only when its version is strictly greater than the stored version, and discard it otherwise, counting the discard as a stale-replay metric.

Records derived from a full file inherit that file’s as-of watermark T as their version, never their processing time. A snapshot therefore cannot overwrite a change that occurred after T. The internal event bus partitions by entity key so that events for one entity are never processed concurrently.

If the provider does not expose a usable monotonic version for a given entity type, that entity type cannot be safely multi-sourced and must be restricted to a single channel until it can be (DI-01, DI-02).

### Consequences

- 
- 
- 
-



Snapshot rule: every record derived from a full file inherits that file's as-of watermark T as its version, never its processing time — so a snapshot can never overwrite a CDC event that occurred after T.

Figure — Version and ordering model: monotonic per-entity versioning resolves the snapshot/CDC race

Figure 2 — Version and ordering model (ADR-003)

## ADR-004 — Degraded-mode data classification

**Status:** Accepted

### Context

ADR-002 makes every read potentially stale. Applied uniformly, the requirement to “remain available regardless of provider availability” means serving a four-hour-old balance as though it were current, with no indication to the customer. In a books and records context that is a misrepresentation, and for figures a customer or a downstream system will act on financially it is worse than being unavailable.

Availability is therefore not unconditionally the correct behaviour, and the requirement as originally stated is wrong. It is replaced by a per-data-subject policy.

### Options considered

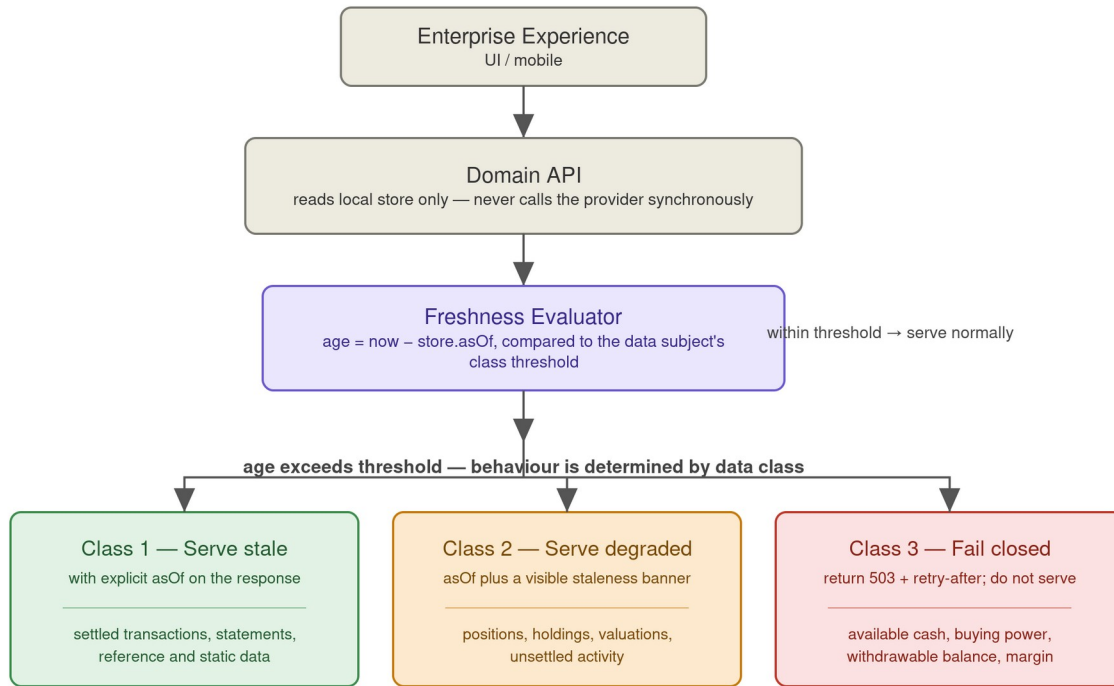
| Option                                                   | Assessment                                                                                                                                                         |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Serve stale silently (Revision 1.0 as written)           | Rejected. Maximises measured availability while creating the largest customer and regulatory exposure in the design.                                               |
| Fail closed on any staleness                             | Rejected. Discards the entire benefit of ADR-002 for data where staleness is harmless, such as settled transactions and reference data.                            |
| Per-data-subject classification with explicit thresholds | Selected. Matches behaviour to consequence, and makes the trade-off a documented decision per data subject rather than an emergent property of the implementation. |

### Decision

Every data subject is assigned a class and a staleness threshold at design time. Every domain API response carries an asOf timestamp and a freshness indicator derived from the envelope watermark, and enterprise experiences are required to surface it. Any data subject not explicitly classified defaults to Class 3 — the design fails safe rather than fails available.

### Consequences

- 
- 
- 
-



Every data subject is assigned a class and a threshold at design time. Unclassified subjects default to Class 3.  
Serving an unmarked stale balance is a misrepresentation, not a resiliency win — availability is not unconditionally correct.

Figure — Read path availability and degraded-mode data classification

Figure 3 — Read path and degraded-mode classification (ADR-004)

| Class              | Behaviour past threshold                           | Proposed threshold    | Example subjects                                                                           |
|--------------------|----------------------------------------------------|-----------------------|--------------------------------------------------------------------------------------------|
| 1 — Serve stale    | Serve with explicit asOf on the response           | <b>24 h</b> (PH-24)   | Settled transactions, statements, reference and static data                                |
| 2 — Serve degraded | Serve with asOf plus a visible staleness indicator | <b>15 min</b> (PH-25) | Positions, holdings, valuations, unsettled activity                                        |
| 3 — Fail closed    | Return 503 with retry-after; do not serve a value  | <b>60 s</b> (PH-26)   | Available cash, buying power, withdrawable balance, margin — anything acted on financially |

## ADR-005 — Transactional outbox with sticky-pending dispatch

**Status:** Accepted

### Context

Domain APIs publish to the provider in near real time when it is available, and the request is pended durably when it is not. The domain constructs the provider-shaped payload and commits it, together with an outbound intent, in a single local transaction, so a write is never lost between the domain's own commit and the gateway picking it up.

Revision 1.0 claimed that partitioning the pending queue by entity key preserved per-entity ordering. It preserves order within the queue, but not across the direct-send / pending boundary. Concretely: the breaker opens and writes W2 and W3 for entity E pend; the breaker half-opens and the drain begins replaying W2; a new write W4 for E arrives, observes a healthy breaker and goes direct; W4 reaches the provider before W3. The provider's record of E is now wrong, and nothing detects it until the next EOD reconciliation.

### Options considered

| Option                                                     | Assessment                                                                                                                                                    |
|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Accept the reordering and repair at reconciliation         | Rejected. Knowingly writing incorrect data to the system of record and relying on a nightly job to notice is not an acceptable posture for books and records. |
| Global pause of direct send until the queue is fully empty | Correct but far too blunt: one slow entity blocks every unrelated write to that LOB instance.                                                                 |
| Per-entity sticky-pending flag                             | Selected. Constrains ordering exactly where ordering matters and nowhere else.                                                                                |

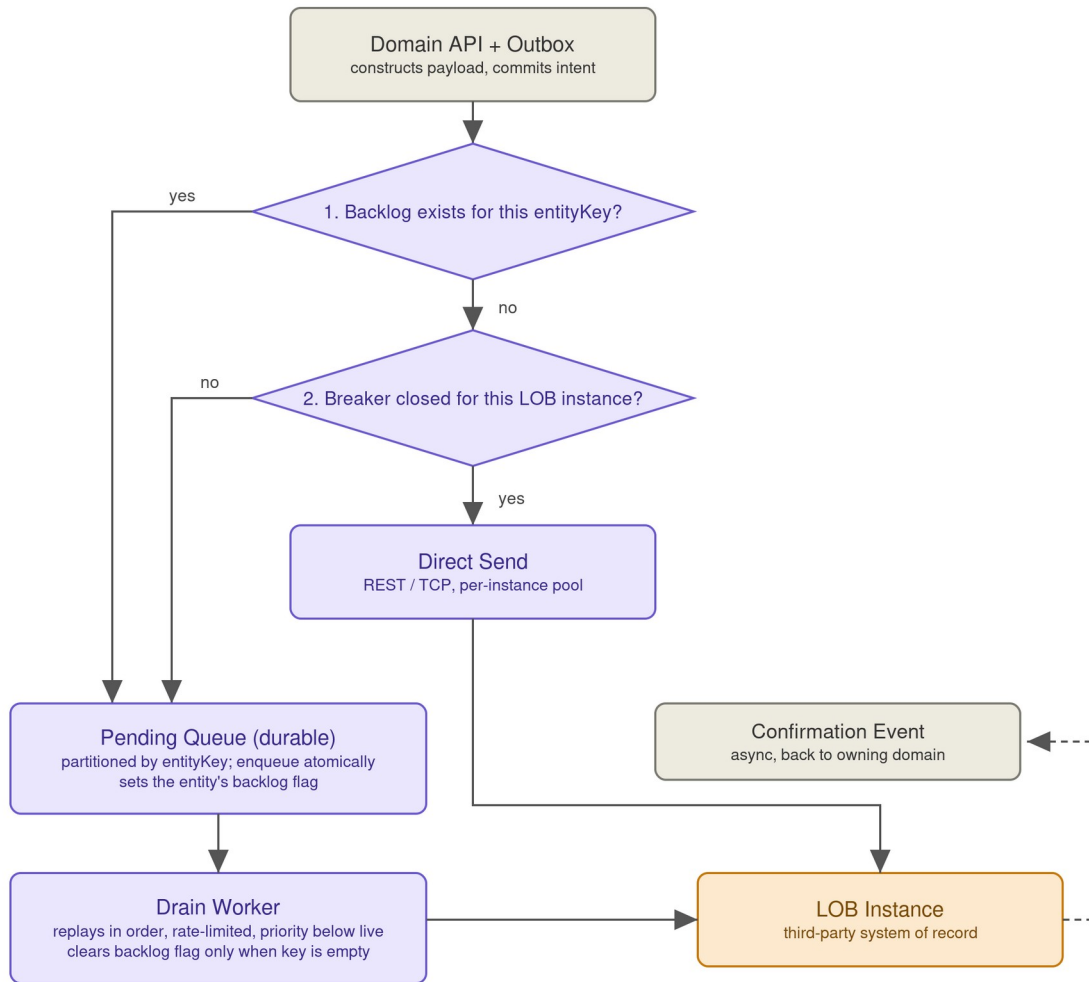
### Decision

The dispatcher checks for an existing backlog on the entity key before it checks breaker state. Once any intent for an entity is pended, every subsequent intent for that entity is pended — regardless of breaker state — until that entity's backlog is fully drained. The drain worker clears the flag only when the key is empty. Enqueue and flag-set are atomic.

The drain replays with backoff and rate limiting rather than as a burst; a provider instance that has just recovered is the worst possible moment to deliver the full backlog at once. Drain traffic runs at a lower priority class than live traffic (§9.3).

### Consequences

- 
- 
- 
-



The backlog check precedes the breaker check. Once any intent for an entity is pended, every subsequent intent for that entity pends until its backlog drains — closing the reordering window that a half-open breaker would otherwise create.

Figure — Corrected outbound dispatch: sticky-pending preserves per-entity order across breaker transitions

Figure 4 — Corrected outbound dispatch with sticky-pending (ADR-005)

## ADR-006 — Per-operation idempotency classification

**Status:** Accepted

### Context

Retries and replays are unavoidable in this design. Applying a single uniform idempotency strategy across all operations is either unsafe (resending an additive operation) or unnecessarily expensive (tracking every replace-style upsert).

### Options considered

| Option                                                        | Assessment                                                                                                                                 |
|---------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Uniform gateway-side dedup for everything                     | Safe but wasteful, and puts a send-log write on the hot path for operations that do not need it.                                           |
| Rely on provider idempotency for everything                   | Unsafe unless the provider guarantees it for every operation, which is the subject of DI-01.                                               |
| Classify per operation, classification supplied by the domain | Selected. The provider's API contract determines the correct strategy per operation, and the domain is the party that knows that contract. |

### Decision

Idempotency handling is classified per operation as domain-supplied metadata attached to each command type, driven by the provider's actual API contract. The gateway executes the classification but does not decide it.

Provider-idempotent upserts — natural-key, replace-style operations such as “set position to X” — are safe to resend on ambiguous timeout or replay and require no gateway-side tracking. Gateway-deduped operations — additive or creation-style operations such as posting a trade or booking a transaction — use a send-log keyed by idempotency key with a RESERVED → SENT → ACKED state machine; retries or replays for an already-ACKED key are skipped and the cached result returned. A reaper clears stuck RESERVED entries after a timeout for crash recovery.

### Consequences

- 
- 
- 
-

## ADR-007 — Ordered-stream quarantine in place of naive dead-lettering

**Status:** Accepted

### Context

Revision 1.0 specified dead-letter queues on both inbound consumption and outbound delivery, with alerting. On an ordered per-entity stream, skipping a message and continuing applies every subsequent event for that entity to a wrong base state — the entity is silently corrupted and keeps being served as clean. On the outbound side, a dead-lettered write means the domain store and the provider now disagree, and the domain will continue presenting the un-booked value to customers.

Dead-lettering is correct for unordered work. It is not correct here, and “alert for manual triage” does not address the fact that wrong data continues to be served in the meantime.

### Options considered

| Option                                                         | Assessment                                                                                                          |
|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Skip and continue (Revision 1.0 as written)                    | Rejected. Trades a visible stall for silent corruption, which is strictly worse.                                    |
| Halt the entire partition or stream on any poison message      | Correct but disproportionate: one bad record for one account stops updates for every entity sharing that partition. |
| Quarantine the affected entity, halt only that entity's stream | Selected. Contains the stall to the entity that is actually affected.                                               |

### Decision

Poison-message handling is distinguished from ordered-stream handling. For ordered channels, a message that cannot be processed causes the affected entity to be quarantined and its stream halted at that point; processing continues for all other entities. Quarantined entities are flagged in the domain store and are excluded from — or explicitly degraded in — customer-facing responses, following the same class rules as ADR-004.

On the outbound side, a permanent delivery failure sets a sync-failed state on the local record with the same serving consequence, and raises a break under §12. Transport-level and unparseable-envelope failures, which are not entity-specific, continue to use a conventional DLQ.

### Consequences

- 
- 
- 
-

## ADR-008 — Change-feed abstraction for CDC and Data Exchange

**Status:** Accepted

### Context

The provider's forthcoming Data Exchange channel is expected to carry the same change data as the current SQL Server replication to Kafka path. Building it as a separate integration would duplicate the projector logic in every domain and make cutover a coordinated big-bang event across all of them.

### Options considered

| Option                                                     | Assessment                                                                                    |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Separate integration per channel                           | Rejected. Duplicates domain-side logic and forces a simultaneous cutover across every domain. |
| Alternate implementation of a common change-feed interface | Selected. Cutover becomes a per-LOB configuration change, reversible without code.            |

### Decision

Data Exchange is built as an alternate implementation of the same change-feed interface as CDC, switchable per LOB instance behind a feature flag, and run in shadow mode before cutover.

Shadow mode compares both feeds against each other and against the corresponding EOD full file, measuring per-entity divergence in value, ordering and latency. Cutover requires a defined shadow duration and divergence tolerance, and rollback is a flag flip with no deployment (PH-27, PH-28).

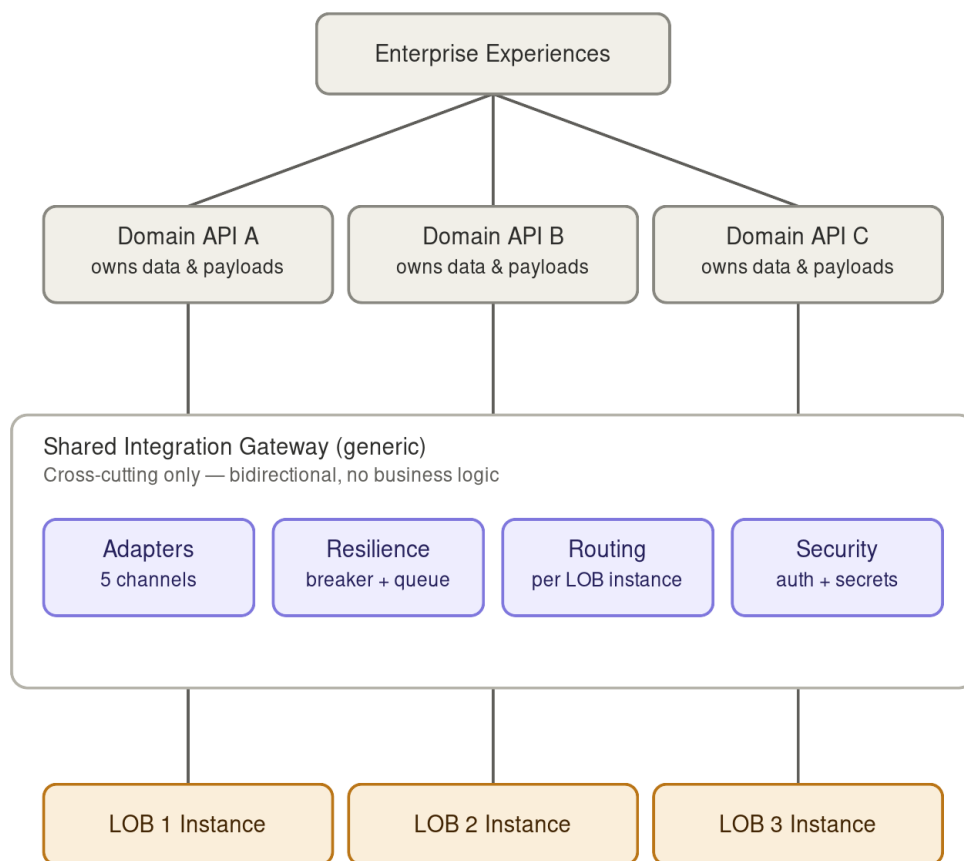
### Consequences

- 
- 
-

## 6. System Overview (Logical View)

Enterprise experiences call domain APIs, which own their data, their local datastores, and — for outbound traffic — the payloads they send to the provider. Domain APIs connect bidirectionally through the shared gateway, which exposes four generic capabilities (channel adapters, resilience, routing and security) and connects bidirectionally to the provider's per-LOB instances.

The figure below is the logical view. It shows one gateway for clarity; the deployment view in Figure 1 is authoritative — the gateway is deployed as one independent cell per LOB instance (ADR-001). Note also that the read path from an enterprise experience does not traverse the gateway at all: reads are served from the domain-local store, which is what makes the availability objectives in §3.2 achievable.



Gray = domain-owned · Purple = generic gateway (bidirectional) · Amber = third-party

Figure 5 — Logical view: generic gateway between domain APIs and the provider

## 7. Inbound Data Path

The provider exposes five distinct channels for the gateway to ingest. Each is normalised by the gateway into a generic delivery envelope — source, LOB instance, entity key, version, asOf and idempotency key — without the gateway interpreting the payload beyond the declarative extraction described in §4.2. Each domain's own consumer then parses, validates and canonicalises the raw payload into its local datastore, applying the version rule from ADR-003.

| Channel                                                | Direction          | Notes                                                                                                                                                                                                                                                                                                                   |
|--------------------------------------------------------|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| EOD files (delta & full)                               | Inbound            | Full files are diffed by the owning domain against current state to synthesise change events, and double as the reconciliation source of truth. Every derived record inherits the file's as-of watermark as its version (ADR-003). Integrity is checked against the file manifest before any record is emitted (DI-03). |
| Near-real-time events (SQL Server replication → Kafka) | Inbound            | Primary low-latency channel; per-LOB consumer groups, per-entity partitioning, quarantine on poison messages (ADR-007), and lag monitoring as a first-class SLI.                                                                                                                                                        |
| REST API                                               | Inbound & outbound | On-demand reads and posts; used for the outbound direct-send path when the provider is available. On-demand reads must still be versioned and passed through the projector rather than written straight to the store, or they reintroduce the ordering race.                                                            |
| TCP/IP                                                 | Inbound & outbound | Likely blocking with proprietary framing; isolated in its own connection pool and bulkhead so it cannot starve REST or Kafka processing.                                                                                                                                                                                |
| Data Exchange (upcoming)                               | Inbound            | Alternate implementation of the same change-feed interface as CDC, switchable per LOB instance behind a feature flag, run in shadow mode before cutover (ADR-008).                                                                                                                                                      |

### 7.1 Envelope wrapper

Wraps every inbound payload, regardless of channel, in the same generic envelope. Full EOD files and delta files are reduced to the same shape: a full file is diffed by the owning domain against its current state to synthesise the equivalent of delta change events, so downstream consumers never need to know whether an update originated from a file or from CDC. What they do need to know — and what the envelope carries — is the version and as-of watermark that determine whether the update should be applied at all.

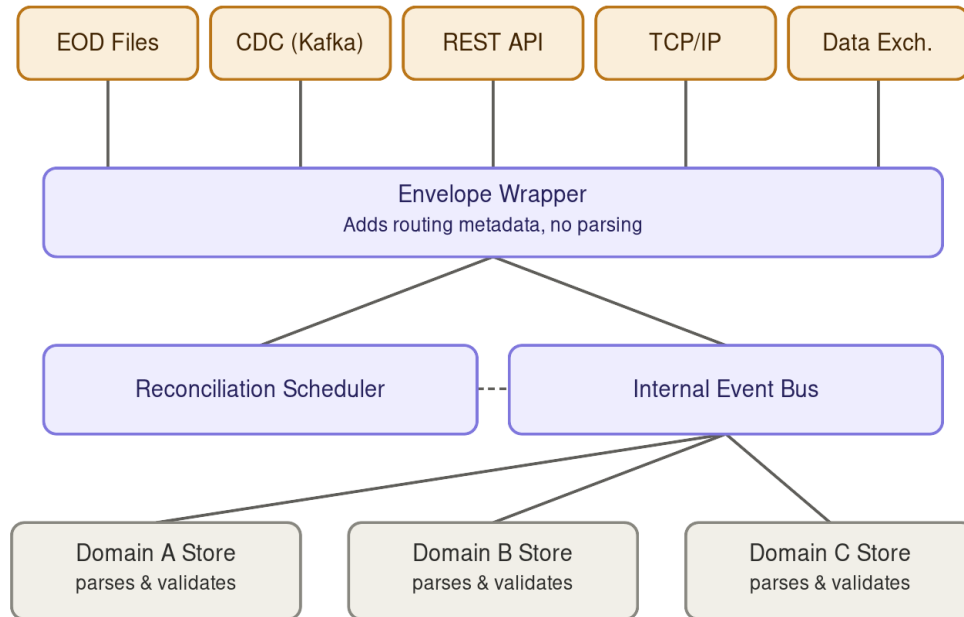
### 7.2 Internal event bus

The shared, generic transport all domain consumers subscribe to. It carries envelopes only. Routing is table-driven against the extracted routing key and requires no interpretation of payload content. Partitioning is by entity key, which is what makes the per-entity ordering guarantee in ADR-003 and the quarantine semantics in ADR-007 possible.

### 7.3 Reconciliation scheduler

Triggers periodic reconciliation runs against full files — the provider's own book of record — but does not perform the comparison itself. It delivers the file and the current event-derived state to the owning domain,

which executes the diff (content-aware work that requires domain knowledge) and publishes any drift result back onto the bus. The operating model around those results is in §12; a drift event with no owner and no resolution SLA is not a control.



Amber = third-party channel · Purple = shared gateway (generic) · Gray = domain-owned

Figure 6 — Inbound path: channel adapters, envelope wrapper and domain-owned projectors

## 8. Outbound Data Path

Domain APIs publish to the provider in near real time when it is available, and the request is pended durably when it is not. The domain constructs the provider-shaped payload and commits it, together with an outbound intent, in a single local transaction — the outbox pattern — so a write is never lost between the domain's own commit and the gateway picking it up. The full decision, including the sticky-pending correction, is recorded as ADR-005 and illustrated in Figure 4.

### 8.1 Outbound dispatcher

Consumes outbound intents and evaluates, in order: whether a backlog exists for the entity key, and only then whether the circuit breaker for the target LOB instance is closed. This is generic delivery logic only — it does not construct or interpret the payload. The order of the two checks is the correction described in ADR-005 and is not an implementation detail.

### 8.2 Direct send, pending queue and drain

If no backlog exists for the entity and the target instance is healthy, the dispatcher sends immediately over REST or TCP. Otherwise the intent is written to the durable pending queue, partitioned by entity key, and the entity's backlog flag is set atomically with the enqueue. The drain worker replays the queue once the provider recovers, in order, with backoff and rate limiting rather than as a burst, and clears the backlog flag for an entity only when that entity's queue is empty.

### 8.3 Confirmation and failure

A successful send, or a final failure after retries are exhausted, is published back as an asynchronous confirmation event. The domain's own consumer updates its local record's sync status; the gateway never writes into a domain's database directly. Permanent failures set a sync-failed state on the local record, which excludes or degrades it in customer-facing responses (ADR-007) and raises a break under §12. The unconfirmed-age obligations in §3.5 apply from the moment the intent is accepted, not from the moment it is dispatched.

## 9. Resiliency, Capacity and Fairness

### 9.1 Failure isolation

- Circuit breakers, bounded retries with backoff, and bulkheads scoped per LOB instance, so one instance's degradation does not affect the others. A global breaker would be wrong here: the instances fail independently.
- TCP/IP has its own connection pool and thread bulkhead, isolated from REST, given its blocking and proprietary framing.
- Cell-based deployment (ADR-001) extends this isolation from the connection layer to the runtime, the datastore and the configuration surface.
- Breaker thresholds are derived from per-instance provider SLAs, which are a blocking discovery item (DI-05). Proposed defaults pending that: open at **50% failures over a 20-request / 30-second window** (PH-31), half-open probe of **1 request every 30 seconds** (PH-32).

### 9.2 Pending queue bounds and load shedding

Revision 1.0 specified a durable pending queue with no bound. At peak write volume a multi-hour provider outage grows it without limit, and an unbounded queue does not fail — it degrades everything around it and then fails in a way that is much harder to reason about.

| Control                               | Threshold                      | Behaviour on breach                                                                                                                                                                     |
|---------------------------------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Queue depth per cell                  | <b>500,000 intents</b> (PH-29) | Stop accepting new intents of the lowest priority class; alert; escalate to business owner. Domain APIs receive an explicit backpressure signal rather than a timeout.                  |
| Oldest intent age per cell            | <b>4 hours</b> (PH-30)         | Escalation under \$3.5; assessment of whether affected write operations should begin failing closed at the domain API rather than continuing to accept intents that cannot be honoured. |
| Backlog depth for a single entity key | <b>1,000 intents</b> (PH-43)   | Alert. Usually indicates a persistently failing entity rather than a provider outage, and should be triaged as a break, not as capacity.                                                |

The important design point is that the system must have a defined behaviour at the limit, and that behaviour must be a deliberate choice between shedding, applying backpressure and failing the user-facing write. Any of the three can be correct for a given operation; having no answer is not.

### 9.3 Priority classes and fairness

Per-instance token-bucket rate limiting prevents one domain's traffic from starving another's access to a shared LOB instance. It does not, on its own, prevent the more likely failure: a large drain backlog consuming the entire bucket and starving live, user-facing traffic against the same instance. Traffic is therefore classified and given guaranteed minimum shares.

| Priority class | Traffic                                          | Guaranteed minimum share                          |
|----------------|--------------------------------------------------|---------------------------------------------------|
| P1 — Live      | User-facing reads and writes in the request path | <b>70%</b> (PH-33)                                |
| P2 — Drain     | Pending-queue replay after recovery              | <b>25%</b> (PH-34)                                |
| P3 — Backfill  | Rehydration, shadow-mode comparison, bulk reload | <b>5%, burstable into unused capacity</b> (PH-35) |

## 9.4 Inbound file processing

Full-file diffing is performed by the owning domain, which is correct on ownership grounds — it is content-aware work. It should be recognised, however, that this is a shifted cost rather than an eliminated one: every domain must build a large-scale diff that completes inside the batch window (PH-07) against files of the size in PH-05.

- The diff must be streaming, not in-memory, at the volumes in PH-05. This should be stated as a requirement on each domain rather than discovered during load testing.
- Live CDC must not be suspended during snapshot application. The version rule (ADR-003) makes interleaving safe, which is a second reason that rule is load-bearing.
- If DI-04 and early sizing show the diff cost is substantial and near-identical across domains, a *generic* streaming diff primitive — keyed record comparison, with the domain supplying key extraction and comparator — belongs in the shared layer after all. This would not violate §4, because the primitive would hold no schema knowledge. Tracked as FU-02.

## 9.5 Backfill, rehydration and disaster recovery

The domain-local projection is derived state, which means it is rebuildable — but only if a rebuild path exists and is fast enough to meet the RTO in §3.4. Kafka retention will not cover a full rebuild, so the rebuild path is the EOD full file plus subsequent change data.

- Onboarding a new domain, recovering from projection loss, and recovering from a regional failure are the same operation and should be built once.
- Rebuild must complete within **4 hours** (PH-36) for the largest domain, and must run at priority class P3 so that it cannot starve live traffic.
- The stateful components — pending queue, send-log, outbox — carry an RPO of zero (§3.4) and require synchronous replication across availability zones. They are the hard part of the DR story and should be designed first, not last.
- Multi-region posture, failover trigger and fallback procedure are not yet decided and are tracked as FU-03.

## 10. Observability and Operations

Revision 1.0 listed useful metrics but defined no service level indicators, no thresholds and no definition of failure. An availability target that cannot be measured cannot be claimed.

### 10.1 Service level indicators

| SLI                       | Measurement                                                                                                         | Objective                                       |
|---------------------------|---------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Read availability         | Proportion of domain API read requests returning a valid response within the latency budget, per data subject class | PH-11 / PH-12                                   |
| Write acceptance          | Proportion of write requests durably committed to the outbox                                                        | PH-13                                           |
| End-to-end freshness      | Provider commit timestamp to domain-store visibility, p99, per channel and per LOB                                  | PH-15                                           |
| Confirmation latency      | Intent accepted to provider ACK, p99                                                                                | PH-14                                           |
| Unconfirmed backlog age   | Age of oldest unconfirmed intent, per cell                                                                          | PH-21 / PH-22 / PH-23                           |
| Quarantine rate and depth | Entities quarantined per interval; total quarantined                                                                | <b>alert above 100 entities / 5 min</b> (PH-41) |
| Reconciliation drift      | Count and value of breaks detected per run                                                                          | Trend; §12                                      |

### 10.2 Operational metrics

- Correlation IDs propagate from inbound envelope through domain processing and back through outbound confirmation, and are carried on every log line and every break record.
- Per-LOB-instance circuit breaker state, pending queue depth and age, per-entity backlog depth, Kafka consumer lag, stale-replay discard rate, and quarantine depth are first-class metrics with dashboards per cell.
- The stale-replay discard rate deserves particular attention: a sudden rise means either a legitimate backfill or a source of out-of-order data that was not anticipated, and the two need to be distinguishable.

### 10.3 Error budgets

Availability and freshness carry separate error budgets, because a system that is up but stale and a system that is fresh but down fail customers in different ways and warrant different responses. Budget consumption is reviewed per cell, since cells fail independently.

### 10.4 Planned provider maintenance

The provider's planned maintenance windows are registered in a maintenance calendar held by the gateway. During a registered window the affected cell suppresses breaker-based paging, pre-throttles the drain so that recovery does not coincide with a burst, and excludes the period from the freshness error budget. Read availability is not excluded — the design is expected to survive planned provider downtime without customer impact, and if it does not, that is a finding.



## 11. Security, Entitlements and Records

Revision 1.0 covered transport security and little else. The concerns below are the ones that determine whether this design is approvable in a regulated firm.

### 11.1 Authentication and credential lifecycle

- Per-LOB-instance credentials (OAuth2 client-credentials or mTLS) held in a secrets vault, scoped to a single cell (ADR-001). No component holds credentials for more than one LOB.
- Rotation must be possible without downtime, and must be exercised on a schedule rather than only in an incident. Certificate expiry is monitored with alerting at **30 days** (PH-39) — expired certificates on integrations of exactly this shape are a leading cause of self-inflicted outages, and they are entirely preventable.
- Credential compromise procedure — revoke, rotate, and replay affected intents — is documented per cell.

### 11.2 Entitlements and information barriers

A shared component that handles data from every line of business raises a question Revision 1.0 did not ask: whether any regulatory or internal information barrier exists between those lines of business, and whether a shared runtime breaches it. In firms where such barriers apply, this is not a theoretical concern.

- Cell-based deployment removes any single process that spans LOBs, which is the primary mitigation.
- The internal event bus must enforce per-LOB authorisation on both publish and subscribe. A domain must not be able to subscribe to an LOB's envelopes merely by knowing the topic name.
- Entitlement decisions on data content remain with the owning domain, which is the only party that can make them, but the gateway must not undermine them by exposing raw envelopes broadly.
- Confirmation with Compliance that no barrier is breached is a blocking item (DI-07).

### 11.3 Data protection

Revision 1.0 specified field-level encryption for sensitive financial data in transit and at rest. In transit this is largely redundant with TLS and adds cost; at rest it is valuable but has a consequence that needs stating — deterministic encryption, which is what would be required to diff or reconcile an encrypted field, is substantially weaker than randomised encryption. Asserting field-level encryption without resolving that is not a security control, it is a statement of intent.

- TLS 1.2 or above, mutual where the provider supports it, on every provider connection and on the internal bus.
- Encryption at rest for the projection stores, pending queue, send-log and outbox, with keys in a managed KMS or HSM.
- Field-level treatment decided per field and recorded: tokenised, deterministically encrypted (with the weakening acknowledged and justified), randomly encrypted, or protected by transport and at-rest encryption alone. Fields required for reconciliation cannot be randomly encrypted, and that constraint should drive the classification rather than being discovered later.

## 11.4 Records retention and immutability

If the data flowing through this integration constitutes books and records, retention obligations attach — potentially to the domain projection stores, the send-log and the pending queue, not only to the provider's own systems. This was absent from Revision 1.0 and is the item most likely to require rework if it is established late.

- Determination of which stores constitute records under SEC Rule 17a-4 and FINRA Rule 4511 is a blocking item for Legal and Compliance (DI-08). Retention pending that determination is assumed to be **6 years, first 2 readily accessible** (PH-38). This is an assumption, not advice, and must be confirmed.
- If WORM storage or non-rewriteable retention applies, it constrains datastore selection for the affected components, so this must be settled before those choices are made.
- Legal hold — the ability to suspend deletion for a defined scope — must be supported wherever retention applies.
- Audit logging on every publish and consume, with correlation ID, actor, LOB instance and outcome, retained under the same policy.

## 11.5 Log and dead-letter hygiene

- Dead-lettered payloads and quarantined records contain account-level data. Access is restricted to a named operational role, contents are masked in any general-purpose log aggregation, and payload retention in the DLQ is capped at **30 days** (PH-40).
- Payloads are never written to application logs; log lines carry the correlation ID and envelope metadata only. A dead-letter queue full of unmasked account data with broad access is a routine audit finding and is straightforward to avoid at design time.

## 12. Reconciliation and Break Management

Reconciliation is an operational control, not merely a scheduled job. Revision 1.0 described the mechanism — the domain diffs a full file against event-derived state and publishes a drift result — but stopped there. A drift event with no owner, no resolution SLA and no evidence trail will not satisfy an audit and, more practically, will not get fixed.

### 12.1 Operating model

| Element               | Definition                                                                                                                                       |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Trigger               | Scheduled against each EOD full file, per LOB instance, within the batch window.                                                                 |
| Execution             | The owning domain performs the diff. The gateway delivers the file and schedules the run but does not compare — the comparison is content-aware. |
| Break record          | Each difference creates a break with entity key, both values, source channels, versions, detection time, correlation ID and severity.            |
| Ownership             | The owning domain's operations function, with a named accountable role per domain.                                                               |
| Resolution SLA        | <b>P1 (customer-visible or financial impact): same business day. P2: 3 business days.</b> (PH-37)                                                |
| Ageing and escalation | Breaks past SLA escalate automatically; an ageing report is reviewed at a defined operational forum.                                             |
| Evidence              | Break records and their resolution are retained under §11.4 as evidence that the control operated.                                               |

### 12.2 Intraday exposure

Reconciliation against EOD files alone leaves drift undetected for up to 24 hours — which is precisely the window during which customers are looking at the data. Detecting at end of day that customers were shown incorrect positions all afternoon is a materially different outcome from detecting it at noon.

- Intraday control totals or per-entity-set checksums, at **hourly** (PH-42) frequency, where the provider can supply them (DI-03).
- Where control totals are unavailable, sampled entity-level verification against the REST API for the highest-risk data subjects, prioritised by Class 3 classification.
- The stale-replay discard rate and quarantine depth act as leading indicators between reconciliation runs and should be monitored as such.

### 13. Blocking Discovery Items

These are not open questions. Each one either determines a design decision that cannot be deferred, or gates a parameter that every downstream threshold depends on. Approval of this document should be conditional on their resolution.

| ID    | Item                                                                                                                    | What it blocks                                                                                                                                                                       | Owner                       |
|-------|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| DI-01 | Does the provider expose a monotonic version, LSN or sequence per entity on each inbound channel?                       | ADR-003 in its entirety. Without it, entities cannot be safely multi-sourced and the multi-channel design is materially constrained. Highest priority.                               | Integration lead + provider |
| DI-02 | Does the provider support idempotent upserts on any create/post operation, or is every write additive?                  | ADR-006 sizing. If nothing is idempotent, every operation takes the send-log path and it is sized for full write volume.                                                             | Integration lead + provider |
| DI-03 | What are the control-total, manifest and checksum conventions on EOD files, and are intraday control totals available?  | Generic file integrity checking (§4.1) and intraday reconciliation (§12.2).                                                                                                          | Integration lead            |
| DI-04 | Channel-to-domain cardinality: does any single channel, file or topic carry entities belonging to more than one domain? | The routing model (§4.2). If not 1:1, table-driven routing alone cannot work and either declarative extraction or fan-out is required — with materially different load implications. | Integration lead + provider |
| DI-05 | What SLA does each LOB instance provide, and does it vary by instance?                                                  | Every breaker threshold, retry and backoff parameter, and drain rate limit in §9.                                                                                                    | Vendor management           |
| DI-06 | Validation that all domain APIs genuinely share the same integration needs.                                             | The shared-layer premise itself (§2). It may well hold, but it is currently an assumption carried forward from the problem statement.                                                | Architecture                |
| DI-07 | Do information barriers exist between the lines of business, and does a shared integration layer breach any of them?    | Section 11.2 and, potentially, ADR-001.                                                                                                                                              | Compliance                  |
| DI-08 | Which stores constitute books and records under SEC 17a-4 / FINRA 4511?                                                 | Section 11.4, and datastore selection for any affected component. Expensive to retrofit.                                                                                             | Legal & Compliance          |
| DI-09 | Confirmed volumetrics and peak profile.                                                                                 | Every placeholder in §3.1, and therefore all capacity-derived parameters.                                                                                                            | Capacity planning           |
| DI-10 | Data Exchange contract: same entity keys, versions and semantics as CDC?                                                | ADR-008. Determines whether the change-feed abstraction can be a true drop-in.                                                                                                       | Integration lead + provider |

## 14. Placeholder Register

Every value in this document that has not been confirmed. These were chosen to make the design reviewable and testable, not because they are known to be right. None may survive into an approved revision.

| ID    | Placeholder value                                     | Used in    | Owner to confirm        |
|-------|-------------------------------------------------------|------------|-------------------------|
| PH-01 | Inbound CDC sustained: 1,200 events/sec               | §3.1       | Capacity planning       |
| PH-02 | Inbound CDC peak: 6,000 events/sec (5× burst)         | §3.1       | Capacity planning       |
| PH-03 | Outbound sustained: 150 intents/sec                   | §3.1       | Capacity planning       |
| PH-04 | Outbound peak: 900 intents/sec                        | §3.1       | Capacity planning       |
| PH-05 | EOD full file: 8M rows / ~4 GB per LOB                | §3.1, §9.4 | Provider                |
| PH-06 | EOD delta file: 200k rows per LOB                     | §3.1       | Provider                |
| PH-07 | Batch window 22:00–04:00 ET, complete by 05:30 ET     | §3.1, §9.4 | Operations              |
| PH-08 | Payload size p50 2 KB / p99 40 KB                     | §3.1       | Provider                |
| PH-09 | Domain API read latency p99 ≤ 200 ms                  | §3.1       | Product + Architecture  |
| PH-10 | Growth allowance 3× over 3 years                      | §3.1       | Business                |
| PH-11 | Read availability market hours 99.95%                 | §3.2       | Business + Architecture |
| PH-12 | Read availability off-hours 99.9%                     | §3.2       | Business + Architecture |
| PH-13 | Write acceptance availability 99.9%                   | §3.2       | Business + Architecture |
| PH-14 | Confirmation p99 ≤ 5 s when provider available        | §3.2       | Provider SLA (DI-05)    |
| PH-15 | Freshness p99 ≤ 5 s on CDC path                       | §3.2       | Business + Architecture |
| PH-16 | Outage definition: >1% errors over 5 min              | §3.3       | Architecture            |
| PH-17 | Projection store RTO 30 min                           | §3.4       | Business                |
| PH-18 | Outbox RTO 15 min, RPO 0                              | §3.4       | Business                |
| PH-19 | Pending queue / send-log RTO 15 min, RPO 0            | §3.4       | Business                |
| PH-20 | Gateway stateless RTO 5 min                           | §3.4       | Architecture            |
| PH-21 | Unconfirmed write: mark pending at 15 min             | §3.5       | Business + Compliance   |
| PH-22 | Unconfirmed write: escalate at 4 h                    | §3.5       | Business + Compliance   |
| PH-23 | Unconfirmed write: treat as break at 24 h             | §3.5       | Business + Compliance   |
| PH-24 | Class 1 staleness threshold 24 h                      | ADR-004    | Business + Compliance   |
| PH-25 | Class 2 staleness threshold 15 min                    | ADR-004    | Business + Compliance   |
| PH-26 | Class 3 staleness threshold 60 s                      | ADR-004    | Business + Compliance   |
| PH-27 | Shadow mode duration before cutover: 20 business days | ADR-008    | Architecture            |
| PH-28 | Shadow divergence tolerance 0.01% of entities         | ADR-008    | Architecture            |

| ID    | Placeholder value                                   | Used in | Owner to confirm           |
|-------|-----------------------------------------------------|---------|----------------------------|
| PH-29 | Pending queue depth cap 500,000 intents per cell    | \$9.2   | Capacity planning          |
| PH-30 | Oldest intent age cap 4 h per cell                  | \$9.2   | Business                   |
| PH-31 | Breaker opens at 50% failures / 20 requests / 30 s  | \$9.1   | Provider SLA (DI-05)       |
| PH-32 | Breaker half-open probe 1 request / 30 s            | \$9.1   | Provider SLA (DI-05)       |
| PH-33 | P1 live traffic guaranteed share 70%                | \$9.3   | Architecture               |
| PH-34 | P2 drain guaranteed share 25%                       | \$9.3   | Architecture               |
| PH-35 | P3 backfill share 5%, burstable                     | \$9.3   | Architecture               |
| PH-36 | Full projection rebuild within 4 h (largest domain) | \$9.5   | Architecture               |
| PH-37 | Break SLA: P1 same business day, P2 3 business days | \$12.1  | Operations + Business      |
| PH-38 | Retention 6 years, first 2 readily accessible       | \$11.4  | Legal & Compliance (DI-08) |
| PH-39 | Certificate expiry alert at 30 days                 | \$11.1  | Security                   |
| PH-40 | DLQ payload retention 30 days                       | \$11.5  | Security + Compliance      |
| PH-41 | Quarantine rate alert above 100 entities / 5 min    | \$10.1  | Operations                 |
| PH-42 | Intraday control totals hourly                      | \$12.2  | Provider (DI-03)           |
| PH-43 | Per-entity backlog alert at 1,000 intents           | \$9.2   | Operations                 |

## 15. Testing Strategy

The value of this architecture lies almost entirely in how it behaves when things fail, so the failure cases have to be tested deliberately rather than encountered in production.

- Contract tests per channel against a provider sandbox, including the envelope extraction expressions, which are configuration and therefore easy to break silently.
- Replay of captured production-shaped traffic for regression, particularly across projector changes.
- Failure injection covering, at minimum: breaker open and half-open transitions; partial drain interrupted by a second outage; duplicate delivery; **out-of-order delivery across channels**; snapshot arriving after newer CDC events; poison message mid-stream; pending queue at its depth cap; and cell loss during drain.
- Load testing at PH-02 and PH-04 peak, including the case where EOD batch processing overlaps with live traffic — that overlap is where the design is most likely to break and is routinely omitted from load tests.
- DR exercises that actually rebuild a projection from files plus change data, and measure it against PH-36 rather than assuming it.
- Acceptance criteria for enterprise experiences that verify asOf and degraded-state rendering, since ADR-004 is only a control if the front end honours it.

## 16. Follow-Up Items

Not blocking approval, but tracked with named owners rather than left implicit.

| ID    | Item                                                                                                                | Owner                  |
|-------|---------------------------------------------------------------------------------------------------------------------|------------------------|
| FU-01 | Cross-cell control plane: aggregate provider quota, fleet-wide configuration validation and staged rollout tooling. | Platform               |
| FU-02 | Assess whether a generic streaming diff primitive belongs in the shared layer (\$9.4).                              | Architecture           |
| FU-03 | Multi-region posture, failover trigger and fallback procedure.                                                      | Platform               |
| FU-04 | Envelope schema versioning and evolution policy.                                                                    | Architecture           |
| FU-05 | Zero-downtime deployment and rollback runbook per cell.                                                             | Platform               |
| FU-06 | Configuration validation, staged rollout and rollback for domain-supplied routing and extraction rules.             | Platform               |
| FU-07 | Break-management tooling and ageing report.                                                                         | Operations             |
| FU-08 | Cost model: provider call volume, licensing, and the operational cost of N cells.                                   | Finance + Architecture |
| FU-09 | Migration and coexistence plan for existing point-to-point integrations.                                            | Architecture           |
| FU-10 | On-call model and runbooks per cell.                                                                                | Operations             |

## 17. Glossary

- **LOB** — line of business; the provider runs one server instance per LOB.

- **Cell** — an independently deployed, independently configured gateway unit serving exactly one LOB instance (ADR-001).
- **Canonical/generic envelope** — the protocol-agnostic wrapper (source, LOB instance, entity key, version, asOf, idempotency key, correlation ID) the gateway attaches to every message, independent of payload content.
- **Version** — a monotonic, provider-sourced ordering value per entity. The basis of conflict resolution (ADR-003).
- **asOf** — the point in time to which a value corresponds. Carried on the envelope and surfaced on every domain API response (ADR-004).
- **Watermark** — the as-of time of a full-file snapshot, inherited as the version by every record derived from it.
- **Outbox pattern** — committing a business change and its outbound intent in one local transaction so the intent is never lost.
- **Sticky-pending** — the rule that once any intent for an entity is pended, all subsequent intents for that entity are pended until its backlog drains (ADR-005).
- **Quarantine** — halting an individual entity's stream on an unprocessable message and flagging it in the domain store, in place of skipping it (ADR-007).
- **Anti-corruption layer** — a boundary that translates an external system's representation into a neutral form so internal domains are not coupled to the external system's model.
- **Break** — a detected discrepancy between the domain projection and the provider's book of record, managed as an operational control (§12).